

Submission Worksheet

Submission Data

Course: IT114-003-F2025

Assignment: IT114 Module 4 Sockets Part3 Challenge

Student: Alanor S. (ass89)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 10/21/2025 2:11:39 PM

Updated: 10/23/2025 9:14:02 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-module-4-sockets-part3-challenge/grading/ass89>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-module-4-sockets-part3-challenge/view/ass89>

Instructions

- Overview Link: https://youtu.be/_029E_aBTfo
- 1. Ensure you read all instructions and objectives before starting.
- 2. Create a new branch from main called M4-Homework
 - 1. `git checkout main` (ensure proper starting branch)
 - 2. `git pull origin main` (ensure history is up to date)
 - 3. `git checkout -b M4-Homework` (create and switch to branch)
- 3. Copy the template code from here: [GitHub Repository - M4 Homework](#)
 - It includes Sockets Part1, Part2, and Part3. Put all into an M4 folder or similar if you don't have them yet (adjust package reference at the top if you chose a different folder name).
 - Make a copy of Part3 and call it Part3HW
 - Fix the package and import references at the top of each file in this new folder (Note: you'll only be editing files in Part3HW)
 - Immediately record to history
 - `git add .`
 - `git commit -m "adding M4 HW baseline files"`
 - `git push origin M4-Homework`
 - Create a Pull Request from M4-Homework to main and keep it open
- 4. Fill out the below worksheet
 - Each Problem requires the following as you work
 - Ensure there's a comment with your UCID, date, and brief summary of how the problem was solved
 - Code solution (add/commit periodically as needed)
 - Hint: Note how /reverse is handled
- 5. Once finished, click "Submit and Export"
- 6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`

3. git push origin M4-Homework

4. On Github merge the pull request from M4-Homework to main

7. Upload the same PDF to Canvas

8. Sync Local

1. git checkout main

2. git pull origin main

Section #1: (3 pts.) Challenge 1 - Coin Flip

Progress: 100%

≡ Task #1 (3 pts.) - Implement a Coin Flip Command

Progress: 100%

Details:

- **Client** must capture the user entry and generate a valid command per the lesson details
 - Command format must be `/flip`
- **ServerThread** must receive the data and call the correct method on **Server**
- **Server** must expose a method for the logic and send the result to everyone
 - The message must be in the format of `<who> flipped a coin and got <result>` and be from the Server
- Add code to solve the problem (add/commit as needed)

📸 Part 1:

Progress: 100%

Details:

Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from **Client**
 - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from **ServerThread**
 - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from **Server**
 - Should only need to create a new method and pass the result message to `relay()`
4. Show 5 examples of the command being seen across all terminals (2+ Clients and 1 Server)
 1. This can be captured in one screenshot if you split the terminals side by side

```
import java.util.*;
import java.io.*;

public class Client {
    // ... (other code) ...

    public void processClientCommands() {
        // ... (other code) ...
        if (command.startsWith("/flip")) {
            // ... (other code) ...
            flip();
        }
    }

    // ... (other code) ...
}
```

Snippet of relevant code showing solution (with ucid/date comment) from Client

```

1 // ...
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...

```

Snippet of relevant code showing solution (with ucid/date comment) from ServerThread

```

1 // ...
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...

```

Snippet of relevant code showing solution (with ucid/date comment) from Server

```

1 // ...
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...

```

Examples



Saved: 10/23/2025 8:40:07 PM

Part 2:

Progress: 100%

Details:

Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1

<https://github.com/AlCSS442/asa89-IT114/commit/003M4-Homework/M4/Part3HW/Client.java>



URL

<https://github.com/AlCSS442/asa>



URL #2

[https://github.com/AlCSS442/asa89-](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)

[IT114/blob/master-](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)

[Homework/M4/Part3HW/Server.java](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)

URL #3

[https://github.com/AlCSS442/asa89-](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)

[IT114/blob/master-](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)

[Homework/M4/Part3HW/ServerThread.java](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)



[https://github.com/AlCSS442/asa](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)



[https://github.com/AlCSS442/asa](https://github.com/AlCSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)



Saved: 10/23/2025 8:40:07 PM

≡ Part 3:

Progress: 100%

Details:

Briefly explain **how** the code solves the challenge (note: this isn't the same as **what** the code does)

Your Response:

When a user types /flip, the client recognizes it as a command, not a normal message. It packages the command into a structured format ([Constants.COMMAND_TRIGGER, "flip"]) and sends it to the server. On the server side, the client's ServerThread receives the message, finds the "flip" keyword in its switch statement, and calls server.handleFlip(this). The handleFlip() method simulates a coin toss by randomly choosing heads or tails. It then creates a message with the user's ID and result, and broadcasts it to all connected clients. So, the client sends structured commands, the ServerThread routes them, and the Server handles the logic.



Saved: 10/23/2025 8:40:07 PM

Section #2: (3 pts.) Challenge 2 - Private Message

Progress: 100%

≡ Task #1 (3 pts.) - Implement a Private Message Command

Progress: 100%

Details:

- **Client** must capture the user entry and generate a valid command per the lesson details
 - Command format must be `/pm <target id> <message>`
- **ServerThread** must receive the data and call the correct method on **Server**
- **Server** must expose a method for the logic
 - The message must be in the format of `PM from <who>: <message>` and be from the Server
 - The result must only be sent to the original sender and to the receiver/target
- Add code to solve the problem (add/commit as needed)

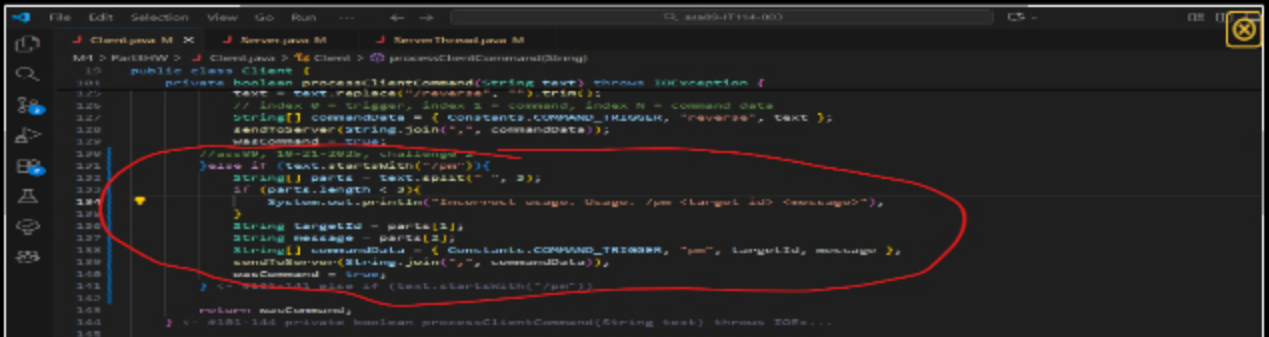
Part 1:

Progress: 100%

Details:

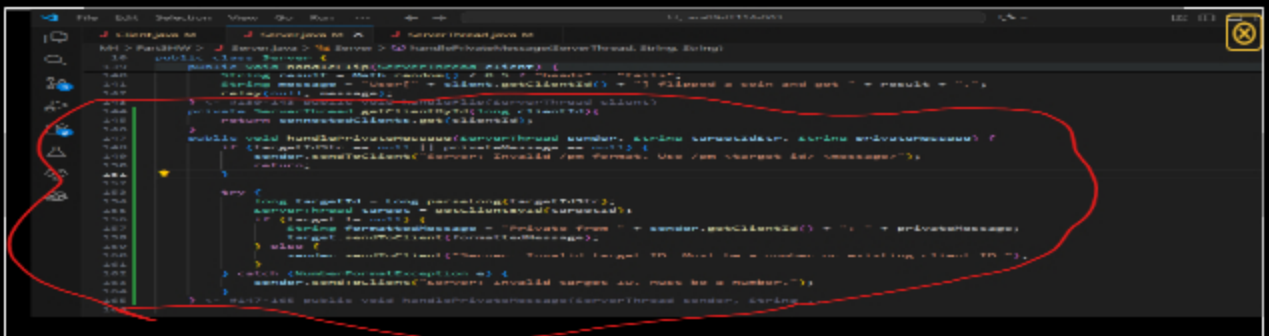
Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from **Client**
 - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from **ServerThread**
 - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from **Server**
 - Should only need to create a new method and send the result message to just the sender and receiver
4. Show 3 examples of the command being seen across all terminals (3+ Clients and 1 Server)
 1. This can be captured in one screenshot if you split the terminals side by side
 2. Note: Only the sender and the receiver should see the private message (show variations across different users)



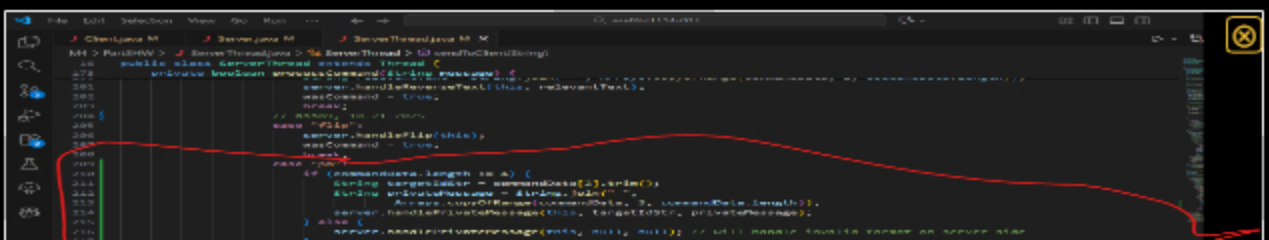
```
10 public class Client {
11     private boolean processClientCommand(String text) throws IOException {
12         text = text.replace("/reverse", "");
13         // Index 0 = trigger, index 1 = command, index 2 = command data
14         String[] commandData = { Constants.COMMAND_TRIGGER, "reverse", text };
15         sendToServer(String.join(" ", commandData));
16         waitCommand();
17
18         // ASCII 10-255, challenge 2
19         // case 1: (text.startsWith("/per")) {
20         String[] parts = text.split(" ", 2);
21         if (parts.length > 1) {
22             System.out.println("Incorrect usage: /per <target id> <message>");
23         }
24         String targetId = parts[1];
25         String message = parts[2];
26         String[] commandData = { Constants.COMMAND_TRIGGER, "per", targetId, message };
27         sendToServer(String.join(" ", commandData));
28         waitCommand();
29         // case 2: (text.startsWith("/per")) {
30         return waitCommand();
31     }
32     // case 3: (text.startsWith("/per")) {
33 }
```

Snippet of relevant code showing solution (with ucid/date comment) from Client



```
10 public class ServerThread {
11     private boolean processCommand(String text) {
12         String result = text.replaceAll("/reverse", "");
13         String message = "user[" + client.getClientId() + "] entered a win and got " + result + "!";
14         sendToClient(message);
15         return waitCommand();
16     }
17
18     private void handleWinMessage(String clientid, String commandData, String privateMessage) {
19         if (clientid != null) {
20             String targetId = commandData[1];
21             String privateMessage = "user[" + client.getClientId() + "] entered a win and got " + privateMessage;
22             sendToClient(targetId, privateMessage);
23         }
24     }
25
26     private void handlePerMessage(String clientid, String commandData, String privateMessage) {
27         if (clientid != null) {
28             String targetId = commandData[1];
29             String privateMessage = "user[" + client.getClientId() + "] entered a win and got " + privateMessage;
30             sendToClient(targetId, privateMessage);
31         }
32     }
33
34     private void handlePerMessage(String clientid, String commandData, String privateMessage) {
35         if (clientid != null) {
36             String targetId = commandData[1];
37             String privateMessage = "user[" + client.getClientId() + "] entered a win and got " + privateMessage;
38             sendToClient(targetId, privateMessage);
39         }
40     }
41 }
```

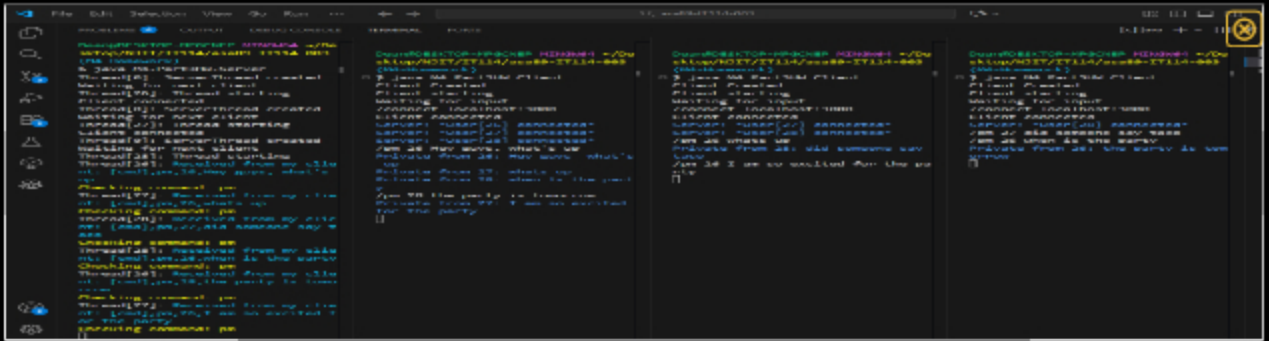
Snippet of relevant code showing solution (with ucid/date comment) from Server



```
10 public class Server {
11     private boolean processCommand(String text) {
12         String result = text.replaceAll("/reverse", "");
13         String message = "user[" + client.getClientId() + "] entered a win and got " + result + "!";
14         sendToClient(message);
15         return waitCommand();
16     }
17
18     private void handleWinMessage(String clientid, String commandData, String privateMessage) {
19         if (clientid != null) {
20             String targetId = commandData[1];
21             String privateMessage = "user[" + client.getClientId() + "] entered a win and got " + privateMessage;
22             sendToClient(targetId, privateMessage);
23         }
24     }
25
26     private void handlePerMessage(String clientid, String commandData, String privateMessage) {
27         if (clientid != null) {
28             String targetId = commandData[1];
29             String privateMessage = "user[" + client.getClientId() + "] entered a win and got " + privateMessage;
30             sendToClient(targetId, privateMessage);
31         }
32     }
33
34     private void handlePerMessage(String clientid, String commandData, String privateMessage) {
35         if (clientid != null) {
36             String targetId = commandData[1];
37             String privateMessage = "user[" + client.getClientId() + "] entered a win and got " + privateMessage;
38             sendToClient(targetId, privateMessage);
39         }
40     }
41 }
```




Snippet of relevant code showing solution (with ucid/date comment) from ServerThread



Outputs



Saved: 10/23/2025 8:44:07 PM

Part 2:

Progress: 100%

Details:

Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1

<https://github.com/AICSS442/asa89-IT1141003M4-Homework/M4/Part3HW/Client.java>



<https://github.com/AICSS442/asa>



URL #2

<https://github.com/AICSS442/asa89-IT1141003M4-Homework/M4/Part3HW/Server.java>



<https://github.com/AICSS442/asa>



URL #3

<https://github.com/AICSS442/asa89-IT1141003M4-Homework/M4/Part3HW/ServerThread.java>



<https://github.com/AICSS442/asa>



Saved: 10/23/2025 8:44:07 PM

Part 3:

Progress: 100%

Details:

Briefly explain **how** the code solves the challenges (note: this isn't the same as **what** the code does)

Your Response:

The /pm code allows one client to send a private message to another through the server. When a user types /pm , the client formats the input as [cmd], pm, , and sends it to the server. The client's ServerThread identifies the "pm" command in its switch statement. It extracts the target ID and message before calling server.handlePrivateMessage(). The server looks up the target client by ID and only forwards the message to that client.



Saved: 10/23/2025 8:44:07 PM

Section #3: (3 pts.) Challenge 3 - Shuffle Message

Progress: 100%

≡ Task #1 (3 pts.) - Implement a Shuffle Message Command

Progress: 100%

Details:

- **Client** must capture the user entry and generate a valid command per the lesson details
 - Command format must be `/shuffle <message>`
- **ServerThread** must receive the data and call the correct method on **Server**
- **Server** must expose a method for the logic and send the result to everyone
 - The message must be in the format of `Shuffled from <who>: <shuffled_message>` and be from the Server
- Add code to solve the problem (add/commit as needed)

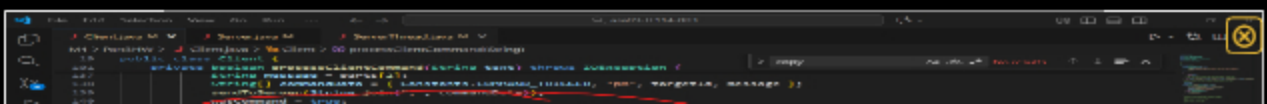
📁 Part 1:

Progress: 100%

Details:

Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from **Client**
 - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from **ServerThread**
 - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from **Server**
 - Should only need to create a new method and do similar logic to `relay()`
4. Show 3 examples of the command being seen across all terminals (2+ Clients and 1 Server)
 1. This can be captured in one screenshot if you split the terminals side by side



URL #1

[https://github.com/AICSS442/asa89-](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Client.java)

[IT114/blob/master-](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Client.java)

[Homework/M4/Part3HW/Client.java](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Client.java)



URL

[https://github.com/AICSS442/asa](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Client.java)



URL #2

[https://github.com/AICSS442/asa89-](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)

[IT114/blob/master-](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)

[Homework/M4/Part3HW/Server.java](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)



URL

[https://github.com/AICSS442/asa](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/Server.java)



URL #3

[https://github.com/AICSS442/asa89-](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)

[IT114/blob/master-](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)

[Homework/M4/Part3HW/ServerThread.java](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)



URL

[https://github.com/AICSS442/asa](https://github.com/AICSS442/asa89-IT114/blob/master/Homework/M4/Part3HW/ServerThread.java)



Saved: 10/23/2025 8:49:04 PM

⇒ Part 3:

Progress: 100%

Details:

Briefly explain **how** the code solves the challenges (note: this isn't the same as **what** the code does)

Your Response:

When a user types /shuffle someText, the client packages it into a structured command (COMMAND_TRIGGER, shuffle, someText) and sends it to the server. The ServerThread reads the message and through a switch case in processCommand(), identifies the "shuffle" command, extracts the text, and calls Server.handleShuffle(). The server then performs a shuffle on the characters and sends the shuffled result to all the clients. So, the client triggers the command, ServerThread sends it, and the server carries out the shuffle logic.



Saved: 10/23/2025 8:49:04 PM

Section #4: (1 pt.) Misc

Progress: 100%

≡ Task #1 (0.33 pts.) - Github Details

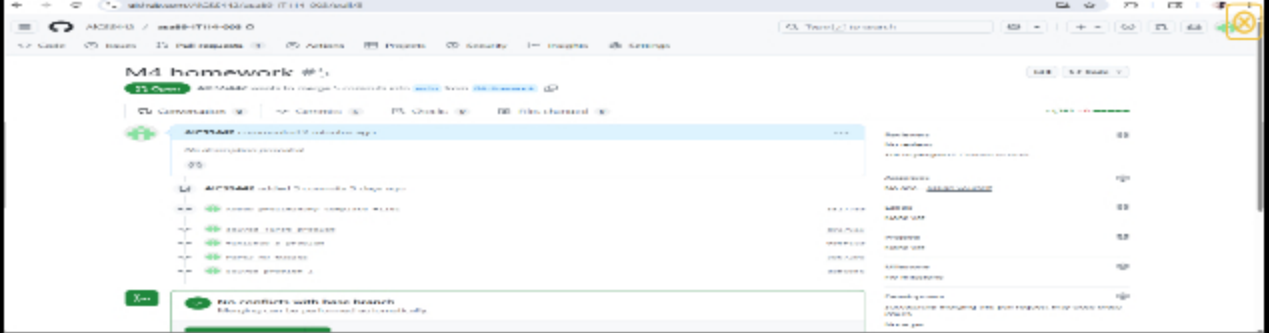
Progress: 100%

📄 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history Following minimum should be present



Pull request



Saved: 10/23/2025 1:10:04 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/AICSS442/asa89-IT114-003>



URL

<https://github.com/AICSS442/asa>



Saved: 10/23/2025 1:10:04 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary





part 2



Saved: 10/23/2025 1:13:22 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned a lot in general about client-server communication through the /shuffle part of the assignment. Essentially, the client sends a structured command, which the ServerThread reads and routes to the server, where the Server executes the actual shuffle logic. I also saw how a server is able to handle multiple clients simultaneously with multithreading. Each client connection runs in its own ServerThread.



Saved: 10/23/2025 8:57:02 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

One part that was easier to implement was the pm command in the Client logic

One part that was easier to implement was the `pin` command in the client logic. Once the command format was defined, it was relatively straightforward to parse the input, extract the target client ID and message, and package it into a structured command. The rest of the commands generally followed similar structure as well.



Saved: 10/23/2025 9:14:02 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I think the hardest part for me was probably the initial part, where I was trying to understand the relationship between Server, Client, and ServerThread and establishing communication between them. This was completely new to me, but once I was able to gain that conceptual understanding, I could proceed. Understanding that the Server listens for incoming connections, each ServerThread manages communication with one specific client. The Client has to handle its own input and output streams to talk with the server, which was initially confusing. I also needed to understand how commands from the client are packaged, sent, and processed on the server side. But once I could visualize this flow, it made more sense. It was definitely a learning process.



Saved: 10/23/2025 9:13:24 PM